

Двоичное дерево

```
struct Node {  
    int data;  
    Node* left, right;  
};
```

Основной способ работы - **рекурсия**

Основные операции

- Поиск
- Подсчёт
- Обход
- Добавление/удаление

Поиск в дереве

```
bool treeSearch(Node* tree, int elem) {  
    if (tree == nullptr) return false;  
    if (tree->data == elem) return true;  
    return treeSearch(tree->left, elem) || treeSearch(tree->right, elem);  
}
```

Обходы дерева

- Inorder Traversal
Для каждой вершины:
 1. Рекурсивно выводим левое поддерево
 2. Выводим саму вершину
 3. Рекурсивно выводим правое поддерево
- Preorder Traversal
Для каждой вершины:
 1. Выводим саму вершину
 2. Рекурсивно выводим левое поддерево
 3. Рекурсивно выводим правое поддерево
- Postorder Traversal
Для каждой вершины:
 1. Рекурсивно выводим левое поддерево
 2. Рекурсивно выводим правое поддерево
 3. Выводим саму вершину

- Levelorder Traversal

Вывод по глубине вершины (начинаем вывод с корня). Реализация:

1. BFS
2. Каждый раз опускаться на нужную глубину